

Competitive Programming

Section 13: Offline Processing

Competitive Programming Course

2026-07-05

Table of contents

1	Why offline processing matters	3
2	The offline pattern	4
3	Vocabulary	5
4	Coordinate compression	6
5	Fenwick tree in this section	7
5.1	The lowbit idea	7
5.2	Prefix sums	8
5.3	Point updates	9
5.4	Why this is logarithmic	9
5.5	Complete Fenwick skeleton	9
6	Worked example 1: threshold range queries	11
7	Worked example 2: distinct values in ranges	12
8	Answer restoration	13
9	When offline processing is not allowed	14
10	Common offline keys	15
11	C++ implementation notes	16
12	Python implementation notes	17
13	Proof habits	18
14	Common wrong approaches	19
14.1	Forgetting original indices	19
14.2	Moving the pointer backward	19
14.3	Compressing away needed distances	19
14.4	Double-counting old occurrences	19
15	Local judge workflow	20
16	Checklist before submitting	21
17	Exercises	22
17.1	A. Coordinate Compression	22
17.2	B. Range Count At Most	22
17.3	C. Distinct Range Queries	22

1 Why offline processing matters

Most query problems are presented online:

```
Read query 1, answer it.  
Read query 2, answer it.  
...
```

But unless the statement explicitly forces online behavior, you are allowed to read all queries first. Once you know every query, you can reorder the work.

That is **offline processing**.

The tradeoff is simple:

```
Original order is convenient for output.  
Sorted order is convenient for computation.
```

The bridge is storing each query's original index.

2 The offline pattern

Most offline solutions follow this shape:

```
read all input
attach original index to each query
sort queries by a useful key
move a pointer or data structure monotonically
write answers into ans[original_index]
print ans in original order
```

You should think of the sorted order as an implementation detail. The final answers still belong to the original query order.

3 Vocabulary

Online query. A query that must be answered before future queries are known or before future updates happen.

Offline query. A query that may be stored and answered later after reordering.

Coordinate compression. Replacing large values by small ranks while preserving equality and order.

Fenwick tree. A data structure for prefix sums with point updates in $O(\log n)$. This section uses it as a local tool for query processing.

Answer restoration. Writing computed answers to `ans[id]` so they can be printed in original query order.

Monotone pointer. A pointer that only moves forward while processing sorted queries.

4 Coordinate compression

Coordinate compression is the simplest offline transformation.

Given:

```
100 50 100 -5 50 7
```

Distinct sorted values:

```
-5 7 50 100
```

Ranks:

```
-5 -> 0  
7 -> 1  
50 -> 2  
100 -> 3
```

Compressed array:

```
3 2 3 0 2 1
```

Compression preserves:

- equality;
- less-than order;
- number of distinct values.

It does not preserve:

- distances between values;
- sums of values;
- arithmetic relationships.

So compression is safe for indexing, ordering, counting, and equality. It is not safe if the problem needs real coordinate lengths unless you keep the original coordinates too.

5 Fenwick tree in this section

A Fenwick tree stores an array of counts or sums and supports:

```
add(position, delta)
prefix_sum(position)
range_sum(l, r) = prefix_sum(r) - prefix_sum(l-1)
```

In this section, we only use point updates and range-sum queries.

Example:

```
positions: 1 2 3 4 5
active:    0 1 0 1 1
range_sum(2,5) = 3
```

You can think of the Fenwick tree as a compressed prefix-sum array that supports updates efficiently.

That sentence is useful only after you know the mechanics, so here is the minimal Fenwick tree you need for this section.

5.1 The lowbit idea

Fenwick trees are one-based. For an index i , define:

```
lowbit(i) = i & -i
```

This is the largest power of two dividing i .

Examples:

i	binary	$\text{lowbit}(i)$
1	001	1
2	010	2
3	011	1
4	100	4
5	101	1
6	110	2
7	111	1
8	1000	8

Fenwick entry $\text{bit}[i]$ stores the sum of a block ending at i with length $\text{lowbit}(i)$.

For $n = 8$:

```
bit[1] stores a[1..1]
bit[2] stores a[1..2]
bit[3] stores a[3..3]
bit[4] stores a[1..4]
bit[5] stores a[5..5]
bit[6] stores a[5..6]
bit[7] stores a[7..7]
bit[8] stores a[1..8]
```

Diagram:

```
index:  1 2 3 4 5 6 7 8
bit[1]: [1]
bit[2]: [1 2]
bit[3]:      [3]
bit[4]: [1 2 3 4]
bit[5]:          [5]
bit[6]:          [5 6]
bit[7]:              [7]
bit[8]: [1 2 3 4 5 6 7 8]
```

5.2 Prefix sums

To compute `prefix_sum(i)`, repeatedly take the block ending at `i`, then jump left by that block's size:

```
sum = 0
while i > 0:
    sum += bit[i]
    i -= lowbit(i)
```

Example for `prefix_sum(7)`:

```
take bit[7] -> covers [7..7]
move to 6
take bit[6] -> covers [5..6]
move to 4
take bit[4] -> covers [1..4]
move to 0
```

Together these blocks cover exactly `[1..7]`.

```
[1..4] + [5..6] + [7..7] = [1..7]
```

5.3 Point updates

To add `delta` at position `i`, update every Fenwick block that contains `i`. Those block endpoints are reached by repeatedly adding `lowbit(i)`:

```
while i <= n:
    bit[i] += delta
    i += lowbit(i)
```

Example: add at position 5 when `n = 8`.

```
i = 5: bit[5] covers [5..5]
i = 6: bit[6] covers [5..6]
i = 8: bit[8] covers [1..8]
```

All three blocks contain position 5, so all three must change.

5.4 Why this is logarithmic

Each jump changes one binary block of the index. There are only $O(\log n)$ such blocks before the index reaches 0 for a query or exceeds `n` for an update.

So:

```
add:           $O(\log n)$ 
prefix_sum:    $O(\log n)$ 
range_sum:     $O(\log n)$ 
```

The range query is just:

```
range_sum(l, r) = prefix_sum(r) - prefix_sum(l - 1)
```

That is why Fenwick trees fit offline processing well: sorted queries decide which positions are active, and the Fenwick tree quickly counts active positions inside a requested index range.

5.5 Complete Fenwick skeleton

C++:

```
struct Fenwick {
    vector<int> bit;

    explicit Fenwick(int n) : bit(n + 1, 0) {}

    void add(int idx, int delta) {
        while (idx < (int)bit.size()) {
            bit[idx] += delta;
        }
    }
};
```

```

        idx += idx & -idx;
    }
}

int sum(int idx) const {
    int total = 0;
    while (idx > 0) {
        total += bit[idx];
        idx -= idx & -idx;
    }
    return total;
}

int range_sum(int left, int right) const {
    return sum(right) - sum(left - 1);
}
};

```

Python:

```

class Fenwick:
    def __init__(self, n: int) -> None:
        self.bit = [0] * (n + 1)

    def add(self, idx: int, delta: int) -> None:
        while idx < len(self.bit):
            self.bit[idx] += delta
            idx += idx & -idx

    def sum(self, idx: int) -> int:
        total = 0
        while idx > 0:
            total += self.bit[idx]
            idx -= idx & -idx
        return total

    def range_sum(self, left: int, right: int) -> int:
        return self.sum(right) - self.sum(left - 1)

```

For now, treat Fenwick trees as a focused tool for dynamic prefix sums. Later data-structure sections can go deeper; Section 13 only needs this point-update, range-query version.

6 Worked example 1: threshold range queries

Array:

```
index: 1 2 3 4 5
value: 4 1 7 3 4
```

Queries:

```
id  l r x   asks
0   1 5 4   count values <= 4 in [1,5]
1   2 4 2   count values <= 2 in [2,4]
2   3 5 10  count values <= 10 in [3,5]
```

Sort array positions by value:

```
(1,2), (3,4), (4,1), (4,5), (7,3)
```

Sort queries by x:

```
x=2, id=1
x=4, id=0
x=10, id=2
```

Process:

```
x=2:
  add position 2
  answer id 1 = active in [2,4] = 1

x=4:
  add positions 4, 1, 5
  answer id 0 = active in [1,5] = 4

x=10:
  add position 3
  answer id 2 = active in [3,5] = 3
```

The query order changed, but the answer array restores it:

```
id 0 -> 4
id 1 -> 1
id 2 -> 3
```

7 Worked example 2: distinct values in ranges

Array:

```
index: 1 2 3 4 5 6 7
value: 1 2 1 3 2 4 1
```

Suppose we process positions from left to right. For every value, keep only its latest occurrence active.

After processing position 5, active positions are:

```
value 1 latest at 3
value 2 latest at 5
value 3 latest at 4
```

Active marker array:

```
index: 1 2 3 4 5
mark:  0 0 1 1 1
```

For a query $[2,5]$, the number of distinct values is:

```
sum marks in [2,5] = 3
```

Why does this work? For every value that appears in $[1, r]$, its latest occurrence among positions $1..r$ is inside $[1, r]$. Values that do not appear in $[1, r]$ have no active marker inside that range.

8 Answer restoration

Never print while processing sorted queries unless sorted order is the requested output order.

Attach an id:

```
query 0: l=1 r=5 x=4
query 1: l=2 r=4 x=2
query 2: l=3 r=5 x=10
```

After sorting:

```
(x=2, id=1)
(x=4, id=0)
(x=10, id=2)
```

Store:

```
ans[id] = computed answer
```

Then print:

```
ans[0], ans[1], ans[2]
```

9 When offline processing is not allowed

Do not use offline processing when:

- each answer changes future input or future decisions;
- the statement explicitly says queries must be answered online;
- there are updates and queries interleaved in a way your sorting would change time order;
- the judge is interactive.

Offline processing is safe when queries are independent questions about fixed data, or when updates can also be represented in a sorted/event order that preserves their meaning.

10 Common offline keys

Query shape	Sort key	Data structure
count values $\leq x$ in range	threshold x	Fenwick over positions
distinct values in $[l, r]$	right endpoint r	Fenwick over latest occurrences
sum of points with coordinate $\leq x$	coordinate x	running sum
many range queries on static array	block of l , then r	Mo's algorithm
point-in-rectangle queries	x -coordinate event	Fenwick/segment tree over y

11 C++ implementation notes

Store query ids explicitly:

```
struct Query {  
    int l;  
    int r;  
    long long x;  
    int idx;  
};
```

Fenwick tree is usually one-based:

```
void add(int idx, int delta) {  
    while (idx <= n) {  
        bit[idx] += delta;  
        idx += idx & -idx;  
    }  
}
```

For maps of large values, use `unordered_map<long long,int>` or compress values first.

12 Python implementation notes

Tuples make sorting queries concise:

```
ordered = sorted((x, l, r, idx) for idx, (l, r, x) in enumerate(queries))
```

Fenwick tree methods should be small and iterative. Avoid rebuilding prefix sums after every update.

Use lists for answer restoration:

```
ans = [0] * q  
ans[idx] = value
```

13 Proof habits

Offline proofs usually have two parts.

First, prove the sorted order has the right active set:

```
When processing threshold  $x$ , the data structure contains exactly the elements  
with value  $\leq x$ .
```

Second, prove answer restoration:

```
Even though queries are processed in sorted order, ans[id] receives the answer  
for the original query with that id.
```

14 Common wrong approaches

14.1 Forgetting original indices

The computation may be correct but printed in sorted order.

14.2 Moving the pointer backward

Offline sort keys are chosen so a pointer only moves forward. If your pointer needs to move backward, you probably chose the wrong key or need a different algorithm.

14.3 Compressing away needed distances

Compression rank $3 - 1 = 2$ does not mean the original coordinates differ by 2.

14.4 Double-counting old occurrences

For distinct range queries, each value should have exactly one active latest occurrence. When a value appears again, remove its previous marker.

15 Local judge workflow

Run Section 13:

```
python3 sections/13_offline_processing/check.py
```

Run the reference solutions:

```
CP_TARGET=solution python3 sections/13_offline_processing/check.py
```

The random tests compare the offline algorithms against direct brute force on small arrays and query sets.

16 Checklist before submitting

- Can the queries legally be reordered?
- What is the sort key?
- What invariant does the data structure satisfy after moving the pointer?
- Did every query keep its original index?
- Are Fenwick indices one-based?
- Are values too large for array indexing without compression?
- Are answers printed in original order?

17 Exercises

17.1 A. Coordinate Compression

Practice turning large values into stable ranks.

17.2 B. Range Count At Most

Practice sorting queries by threshold and restoring answers.

17.3 C. Distinct Range Queries

Practice sorting by right endpoint and maintaining latest occurrences.